Event-Driven Architecture (EDA) Pattern

Category: Data / messaging **One-liner:** Components communicate by publishing and reacting to events rather than calling each other directly — giving loose coupling, asynchrony, independent scaling, and **real-time** responsiveness (reacting within moments of something happening).

Part 0 — Concepts you need first

What is an "event"?

An **event** is a record that **something happened** — a fact about the past, named in the **past tense**: `OrderPlaced`, `PaymentReceived`, `UserRegistered`. The tense isn't cosmetic: an event is a *notification of something already done*, not a request. Contrast a **command** — an instruction named imperatively (`PlaceOrder`, `ChargeCard`). **A command can be refused; an event cannot** — it already happened.

Synchronous vs. asynchronous

Synchronous — send a request and **wait** for the response (a phone call). **Asynchronous** — send a message and **carry on immediately**, handling any response when it arrives (email). For a server: in a synchronous call the calling **thread** is **blocked and useless** for the whole wait; asynchronously it's free at once. (§ Sync vs async.)

What is a message queue / event bus?

Middleware that accepts messages from senders and delivers them to receivers — **Kafka** (a distributed, durable event log) and **RabbitMQ** (a classic message broker) are common (§ Message queues). Key properties:

- The **sender doesn't need to know who receives** — it publishes to the bus
- Messages are usually **persisted**, so if a receiver is down the message waits
- **Many receivers** can consume the same message independently

What is publish-subscribe?

Publishers send to a named channel ("topic") without knowing who listens; **subscribers** register interest and receive everything published there. **The publisher holds no subscriber list**, so adding a subscriber needs no publisher change — the source of EDA's flexibility.

What is "coupling"?

How much one component depends on knowledge of another.

- **Tight coupling:** A calls `serviceB.doThing()` — A must know B exists, where it lives, its **API** and response format; change B and A may break.
- **Loose coupling:** A publishes `ThingHappened` without knowing who listens — B, five services, or none. **A is unaffected either way.**

Part 1 — The problem

Distributed systems brought **scalability** and **resilience** but new challenges:

- **Managing complex interactions** — many components must communicate effectively, and changes in one part must be reflected in others. (Everyday example: the "your product shipped" or "new blog post" email is a system handling complex interactions efficiently.)
- **The call for asynchronous communication** — synchronous request-and-wait works small but **quickly becomes a bottleneck**: *what happens when thousands or millions of components are all waiting?* Each holds resources doing nothing. Async is far more efficient but **hard to manage well**. (Why can you

keep adding items to a cart while earlier requests are still processing? Asynchronous communication.)

- **The demand for real-time responsiveness** — users expect a status update to reach friends immediately; multiplayer games can't tolerate delay.
- **Data consistency** — with components reading and writing **concurrently**, how do others learn of a change? A banking transaction must update your balance promptly and accurately even while another transaction hits the same account.

Part 2 — The solution

EDA revolves around the **production, detection, consumption of, and reaction to events**. An **event** is a state change produced by an **event producer** and consumed by one or more **event consumers**. (The courier analogy: sender = producer, package = event, recipient = consumer, delivery service = event bus.)

Part 3 — The benefits

Decoupling

Producers and consumers **don't need to know each other exist**, making the system adaptable. One **photo upload** event can independently trigger notifying followers, updating the profile, and alerting moderation if content violates policy.

> The part worth pausing on: **six months later you add "generate a thumbnail"** by writing a new consumer of `PhotoUploaded`. **You never touch the upload code.** With direct calls you'd modify, retest and redeploy the upload service for every new reaction, forever. That difference in change cost is the pattern's real value.

Asynchrony

The producer emits and **carries on without waiting** — **non-blocking**, improving responsiveness and **throughput** (work done per second).

Scalability

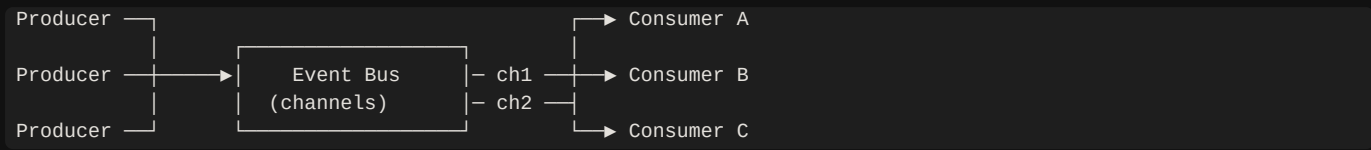
Decoupled producers and consumers **scale independently**: a surge of events → add consumer **instances**, and vice versa.

Real-time processing

Events can be handled **immediately** — essential for **fraud detection in banking** or real-time **analytics**.

Part 4 — The components

Component	Role
Event Producers	Create events. They define what an event is, encapsulate the relevant data, and send it to the Event Bus
Event Bus (message queue)	The backbone. Receives events from producers and transmits them to consumers. **This is what decouples them**
Event Channels	Pathways on the bus. They categorize events by nature, or by the kind of consumers they cater to, so each consumer listens only to the channel relevant to it
Event Consumers	Listen to events via the bus and react. The reaction could be starting a new process, or merely recording the event



Part 5 — Three structural variants

Choose by requirements, complexity and how much control you need.

Variant	How it works	Advantage	Disadvantage
Mediator EDA	Producers send events to a **central mediator** that knows all producers and consumers, decides routing, and performs **aggregation, filtering and transformation** (combining, dropping, reshaping events)	**Control** — complex routing and transformation	The mediator can become a **bottleneck**, limiting scalability
Broker EDA	**No mediator**: producers publish straight to channels; consumers subscribe directly. **The broker *is* the event bus**	**Scalability and simplicity**	Less control over routing and processing
Hybrid EDA	Both — e.g. a mediator for a few complex flows, direct broker delivery for everything else	Mix of **centralized control and high scalability**	Two models to run

---

Part 6 — Inner workings

Event generation and propagation

An event starts with a **change of state** — a user action, a sensor reading, an internal update — packaged as **a bundle of data representing a significant occurrence**, then carried by the **Event Bus** through **Event Channels** that categorize and route by type so the right events reach the right consumers.

Routing

- **Mediator EDA:** a central component routes by event type and known consumers, possibly aggregating, filtering or transforming first.
- **Broker EDA:** no central decision-maker — producers place events in channels, consumers subscribe. Simpler, less controlled.

Event processing and state change

The consumer acts on the event data, **changing system state** — from updating a database value to triggering a physical action in an **IoT** (Internet of Things) device.

How consumers know when to act

Consumers are **always listening** on their subscribed channels and act when an event arrives — usually implemented with the **Observer** (objects register with a subject and are called back on change) or **Publish-Subscribe** patterns.

Real-time illustration — traffic management

Sensors (producers) emit an event per vehicle detected; events flow via the bus to traffic signals (consumers), which **adjust their timing** by vehicle count — changing system state (traffic flow) in real time.

Event Sourcing and CQRS

Event Sourcing stores changes as a **sequence of events**, not just a **snapshot** of current state, giving **full auditability** (proof of what happened when), **versioning**, and **"time-travel"** to any past state. Instead of "balance = £70" you store `Deposited £100`, `Withdrew £30` and **derive** the balance by replaying — how accounting ledgers have always worked.

CQRS separates reads from writes — natural in EDA, where event handlers (writes) are modelled separately from views (reads). Both are covered in depth, including when event sourcing is worth its cost, in CQRS Part 3.

Part 7 — Worked example: traffic management (Java)

The event

```java
public class VehicleEvent {
    private final String vehicleId;
    private final String type;

    public VehicleEvent(String vehicleId, String type) {
        this.vehicleId = vehicleId;
        this.type = type;
    }

    public String getVehicleId() { return vehicleId; }
    public String getType()      { return type; }
}
```

A vehicle 'entering' or 'leaving' a zone. `final` **fields and no setters — the event is immutable**, deliberately: it describes something that **already happened**; letting a consumer modify it is meaningless (you can't change the past) and dangerous (other consumers would see the altered version).

The producer — a sensor

```java
public class Sensor implements EventProducer {
    private EventBus eventBus;

    public Sensor(EventBus eventBus) {
        this.eventBus = eventBus;
    }

    public void vehicleDetected(String vehicleId, String action) {
        VehicleEvent event = new VehicleEvent(vehicleId, action);
        eventBus.publish(event);
    }
}
```

Notice what's absent: the sensor has no reference to any traffic light and doesn't know consumers exist. It publishes and returns — **fire and forget**.

The bus and channels

```
public interface EventBus {
    void publish(VehicleEvent event);
    void addChannel(EventChannel channel);
}

public class SimpleEventBus implements EventBus {
    private List<EventChannel> channels = new ArrayList<>();

    @Override
    public void publish(VehicleEvent event) {
        for (EventChannel channel : channels) {
            channel.add(event);
        }
    }

    @Override
    public void addChannel(EventChannel channel) {
        channels.add(channel);
    }
}

public class SimpleEventChannel implements EventChannel {
    private List<EventConsumer> consumers = new ArrayList<>();

    @Override
    public void add(VehicleEvent event) {
        for (EventConsumer consumer : consumers) {
            consumer.onEvent(event);
        }
    }

    @Override
    public void addConsumer(EventConsumer consumer) {
        consumers.add(consumer);
    }
}
```

Publishing adds the event to every channel; each channel passes it to all its consumers.

The consumer — a traffic light

```
public class TrafficLight implements EventConsumer {
    @Override
    public void onEvent(VehicleEvent event) {
        if ("entering".equals(event.getType())) {
            System.out.println("Vehicle " + event.getVehicleId()
                + " is entering. Adjusting traffic light timings.");
        } else if ("leaving".equals(event.getType())) {
            System.out.println("Vehicle " + event.getVehicleId()
                + " has left. Resetting traffic light timings.");
        }
    }
}
```

What this example simplifies — read before copying it

The roles are right; the mechanics aren't production-shaped:

- **Synchronous and in-process.** `publish` calls consumers **on the caller's thread**, so the sensor waits for every traffic light — destroying the asynchrony that is the main benefit. Real EDA uses Kafka or RabbitMQ with consumers in **separate processes** and the producer returning immediately.
- **No persistence.** If the process dies, **in-flight** events (published, not yet processed) are lost; real buses persist so events survive restarts and can be replayed.

- **No error handling.** One consumer throwing breaks the loop; later consumers never see the event.
- **The bus broadcasts to every channel** instead of routing by type, so channels don't categorize anything here.

Use it to understand **producer, bus, channel, consumer** — not the implementation.

Part 8 — Performance implications and considerations

Scalability

Scale components independently — consumers can't keep up? Add consumers without touching producers. But **keeping parts balanced so none is overwhelmed takes care.**

Processing speed and latency

Async usually boosts performance — producers don't wait. **The flip side is latency** (event-to-handling delay): as volume grows, **the time from production to processing can grow**. The channel choice (synchronous or asynchronous) also affects speed.

Error handling and message durability

When processing fails, you need to **retry, log, or alert an operator**; **message durability** may need **persistent storage** until events are processed. **These mechanisms add complexity and cost performance.**

System complexity

Tracking event flow and debugging are hard in an async, distributed system; you need **special tools and practices**, and every component must agree on event format and meaning.

> ⚠️ **The cost beginners most underestimate.** In a synchronous system a **stack trace** shows what called what. In EDA, causation spans services *and time* — today's bug was caused by an event published twenty minutes ago by a service you weren't watching. You need **correlation IDs** (one identifier threaded through every event in a causal chain) and **distributed tracing** (recording a request's journey across services) from day one, not after your first **incident**. (§ Observability.)

Event schemas and backward compatibility

All components must understand event structure, so events follow a **well-defined schema** (fields, types, which are required). When it changes you need **backward compatibility** so consumers expecting the old schema don't break.

> **Adding an optional field is safe; removing or renaming one is not.** Old consumers ignore unknown fields but break when one they rely on vanishes. **Schema registries** (services storing every schema version and rejecting incompatible changes) enforce this.

Order of events

Events are **asynchronous and may be processed in any order**; dependencies between events need extra machinery to preserve order.

> If `AccountCreated` and `AccountDeleted` arrive swapped, a consumer deletes an account that doesn't exist yet, then creates one that should be gone. Kafka guarantees ordering only *within a partition* (a topic is split into **partitions**, each an ordered log processed in parallel), so choosing the **partition key** (the value deciding an event's partition, e.g. account ID) matters. (§ Event streaming.)

Testing

Unit tests of individual components miss the interactions; you need **integration and end-to-end tests**, which are harder to write and maintain.

None of this should deter you — it's a map of the trade-offs.

Part 9 — Use cases

- **Real-time analytics** — events consumed as generated give instant insight; especially in **finance**, where live market data drives trading decisions.
- **IoT systems** — sensors, smart devices and alarms generate floods of events; EDA's **asynchrony suits them**: a room-temperature rise triggers an event the air conditioning consumes.
- **E-commerce systems** — placing an order triggers events: inventory verifies stock, billing takes payment, shipping handles delivery — **asynchronously and efficiently**.
- **Microservices architecture** — **instead of direct calls (tight coupling), services emit events others consume**, staying loosely coupled and **evolving independently**.

Part 10 — System design example: a ride-sharing app

Uber/Lyft with EDA:

1. **A rider requests a ride** → an event with pickup and destination, **broadcast** to the system
2. **Nearby drivers** consume it
3. **A driver accepts** → an event consumed by the rider's app announcing the match
4. **During the ride** the driver's app streams **current-location** events; the rider's app **updates the map in real time**
5. **The ride ends** → a **ride completion event**
6. **Billing** consumes it, calculates the fare, and **triggers a payment event**

> A ride genuinely *is* a sequence of things that happened. When the domain is already event-shaped, EDA stops being a technical choice and becomes the obvious model.

Part 11 — Common beginner misconceptions

- **"Events are just asynchronous function calls."** A call has one target and expects a return; an event has **zero or many** targets, unknown to the publisher, and expects nothing.
- **"EDA means no more direct calls anywhere."** When you need an immediate answer ("is this user authorized?") a synchronous call is right. EDA is for notification and reaction, not queries.
- **"The event bus makes my system reliable."** It makes it *decoupled*. Reliability needs persistence, retries, **dead-letter queues** (holding areas for messages that keep failing), and **idempotent** consumers on top.
- **"If I publish an event, it definitely gets processed."** Only with **durable** messaging and **acknowledgements** (the consumer confirming it finished). Fire-and-forget over an in-memory bus loses events on restart.
- **"Consumers see each event exactly once."** Most systems guarantee **at-least-once** delivery (every message arrives, possibly more than once), so **duplicates happen** and consumers must be idempotent *(Retry: idempotency)*.
- **"Ordering is guaranteed."** Generally not system-wide — only within a partition or channel, if the **broker** offers even that.

Part 12 — Check your understanding

1. Why are events named in the past tense?
2. What's the change-cost argument for EDA, using the photo upload example?
3. What's the difference between Mediator and Broker EDA, and what does each trade away?
4. Why is debugging harder in EDA, and what two tools address it?
5. Which schema changes are safe and which break consumers?
6. What's wrong with the example's `publish` method, relative to real EDA?

(Answers: 1 — an event records something that already happened and cannot be refused, unlike a command; 2 — adding a new reaction means writing a new consumer without ever modifying the upload service, whereas direct calls require changing and redeploying the producer each time; 3 — Mediator centralizes routing and transformation, gaining control but risking a bottleneck, while Broker publishes straight to channels, gaining scalability but losing routing control; 4 — causation is spread across services and time so there's no single stack trace, addressed by correlation IDs and distributed tracing; 5 — adding optional fields is safe, removing or renaming is not; 6 — it calls consumers synchronously on the producer's thread, so the producer blocks until all consumers finish, eliminating the asynchrony that is the point.)

Part 13 — Interview soundbite (30 seconds)

> "In event-driven architecture components publish events rather than calling each other. A producer publishes a state change to a bus, and any number of consumers subscribe to the channels they care about. The producer doesn't know who's listening — that's the point: one photo-upload event triggers follower notifications, a profile update and moderation, and later you add thumbnails by writing a new consumer without touching the upload code. You also get asynchrony and independent scaling of producers and consumers. Three variants: broker, publishing straight to channels — simple and scalable; mediator, a central router that filters and transforms — more control, potential bottleneck; and hybrid. Ride-sharing is a natural fit — request, acceptance, live location, completion are all events. Costs: no system-wide ordering, so pick partition keys; schemas must evolve carefully because removing a field breaks old consumers; delivery is usually at-least-once, so consumers must be idempotent; publish via an outbox so DB and events can't diverge; and debugging is hard because causation spans services and time — so correlation IDs and distributed tracing from the start. It pairs naturally with event sourcing and CQRS."

Part 14 — Implementation

A. Runnable demo

Full source: `demos/EventDrivenDemo.java`

```
cd demos && ./gradlew runCh11
```

Failure isolation — one consumer throwing must not affect the others:

```
for (Subscriber s : subs) {
    Runnable task = () -> {
        try { s.handler().accept(e); }
        catch (Exception ex) {
            System.out.printf("      [%s] FAILED: %s (isolated)%n", s.name(), ex.getMessage());
        }
    };
    if (async) pool.submit(task); else task.run();
}
```

Idempotent consumer, using a set of seen event IDs:

```
void handle(Event e) {
    String eventId = e.payload().get("eventId");
    if (!processed.add(eventId)) return;      // add() is false if already present
    chargesApplied++;
}
```

B. What you should see

```
=== 1. SYNCHRONOUS bus: producer waits for every consumer ===
  producer blocked for 114ms  <- the sum of all consumers

=== 2. ASYNCHRONOUS bus: producer returns immediately ===
  producer returned in 0ms  <- did NOT wait
      [moderation] FAILED: moderation service unavailable (isolated)

=== 4. Add a NEW consumer - producer code untouched ===
    [bus] publishing PhotoUploaded (cid=cid-3) to 4 subscriber(s)
  >> 4 reactions now. The upload service never changed.

=== 5. At-least-once delivery -> idempotent consumers ===
      [billing]    charge applied (total=1)
      [billing]    duplicate evt-abc ignored
      [billing]    duplicate evt-abc ignored
  charges actually applied: 1 (must be 1)
```

114ms → 0ms is the value of asynchrony as a measured number; section 4 is the architectural argument — subscribers went from 3 to 4 and **the producer's code was never edited**.

C. Production — Kafka

`bootstrap-servers` are the first Kafka machines a client contacts; `acks: all` makes the producer wait until every **replica** (copy on another broker) has the message; an **offset** is a consumer's position in a partition, and **committing** it records how far it has read.

```
spring:
  kafka:
    bootstrap-servers: kafka-1:9092,kafka-2:9092,kafka-3:9092
    producer:
      acks: all                         # wait for all replicas - no silent data loss
      retries: 3
      properties:
        enable.idempotence: true        # broker de-duplicates producer retries
        max.in.flight.requests.per.connection: 5
    consumer:
      group-id: notification-service
      auto-offset-reset: earliest
      enable-auto-commit: false         # commit only AFTER successful processing
      isolation-level: read_committed
```

Producer — the partition key is the important decision (`MDC` is the logging library's per-request context, carrying the correlation ID):

```java
@Service
public class PhotoEventPublisher {
    private final KafkaTemplate<String, PhotoUploaded> kafka;

    public void publish(String photoId, String userId) {
        PhotoUploaded event = new PhotoUploaded(
                UUID.randomUUID().toString(),      // eventId, for consumer de-duplication
                MDC.get("correlationId"),          // trace the causal chain
                photoId, userId, Instant.now());

        // Key = userId -> all events for one user land on one partition,
        // and Kafka guarantees ordering WITHIN a partition.
        kafka.send("photo-uploaded", userId, event);
    }
}
```

Consumer — manual acknowledgement plus idempotency (a **consumer group**, named by `groupId`, shares a topic's partitions among its members):

```java
@KafkaListener(topics = "photo-uploaded", groupId = "thumbnail-service")
public void onPhotoUploaded(PhotoUploaded event, Acknowledgment ack) {
    if (processedEvents.existsById(event.eventId())) {   // at-least-once => duplicates happen
        ack.acknowledge();
        return;
    }
    try {
        thumbnailService.generate(event.photoId());
        processedEvents.save(new ProcessedEvent(event.eventId(), Instant.now()));
        ack.acknowledge();                               // commit only on success
    } catch (TransientException e) {
        // Do NOT ack - Kafka redelivers
        throw e;
    }
}
```

Dead-letter topic (DLT) — where messages go after retries are exhausted:

```java
@Bean
public DefaultErrorHandler errorHandler(KafkaTemplate<Object, Object> template) {
    var recoverer = new DeadLetterPublishingRecoverer(template,
            (record, ex) -> new TopicPartition(record.topic() + ".DLT", record.partition()));
    return new DefaultErrorHandler(recoverer,
            new ExponentialBackOff(1000L, 2.0));   // 1s, 2s, 4s...
}
```

Schema evolution — register schemas so a **breaking change** (one that makes existing consumers fail) is rejected at build time, not discovered in production:

```yaml
spring.kafka.properties:
  schema.registry.url: http://schema-registry:8081
  value.subject.name.strategy: io.confluent.kafka.serializers.subject.TopicNameStrategy
```

Compatibility mode `BACKWARD` means **new consumers can read old events**: adding an optional field passes; removing a field fails the build.

D. Implementation pitfalls

1. Auto-commit enabled. With `enable-auto-commit: true` Kafka commits offsets on a timer — possibly *before* your handler finishes. Crash mid-processing and the message is **silently lost**. Commit manually after success.

2. Consumers that aren't idempotent. At-least-once makes duplicates certain, not hypothetical. Store processed event IDs.

3. No partition key, then expecting ordering. Kafka orders **within a partition only**; without a key, events for one **entity** (e.g. one account) scatter and arrive out of order. Key by entity ID.

4. Publishing before committing the database transaction. Write to the DB, then publish, crash in between: state changed, nobody told. Use the **Transactional Outbox** — explained in Saga 13D.

5. Events carrying too much or too little. Too little and every consumer calls back to the producer, recreating coupling; too much and the event becomes a schema everyone depends on. Carry **identifiers plus the facts that actually changed**.

6. Removing or renaming event fields. Old consumers break. Additive changes only, enforced by a schema registry.

7. No correlation ID. You can't reconstruct what caused what. Generate at the edge, propagate through every event, log on every line.

8. No dead-letter queue. A permanently failing message blocks its partition forever, stalling everything behind it. Always configure a DLT with alerting.

Related patterns

- **CQRS** — separate read/write models, commonly built on EDA
- **Saga** — choreography sagas are EDA applied to distributed transactions
- **Retry** — requeueing and dead-letter queues are retry in an async context
- **Bulkhead** — a queue between components is a natural isolation boundary